

1. Frontend (React) Technical Points Covered

React fundamentals

- React project setup using **Vite**
- Functional components
- JSX structure and component-based UI design
- Reusable components
- Props and component composition
- State management using `useState`
- Side effects and API calls using `useEffect`
- Event handling in forms, buttons, table actions, etc.
- Conditional rendering
- Controlled form components
- Form reset and edit-mode data binding
- Loading and error handling in UI

React routing

- Routing using **React Router**
- Public routes and protected routes
- Navigation after login/logout
- Route-level authorization handling
- Redirecting unauthenticated users to login page

React forms / CRUD UI

- Add Employee form
- Edit Employee form
- Delete confirmation flow
- Search input / filter form
- Paging UI
- Sorting UI
- Validation message display
- Form submission to API
- Handling create/update mode in same component

React architecture / code organization

- Separation of:
 - pages
 - components
 - services
 - utilities
 - route guards
- API service layer using **Axios**

- Token storage helper
- reusable auth helper functions

React authentication implementation

- Login page
- Register page
- JWT token storage
- Refresh token storage
- Logout flow
- Protected route implementation
- Role-based access handling on frontend if added

Axios / API integration

- Axios instance creation
- Global baseUrl
- Request interceptor for attaching JWT token
- Response interceptor for handling 401
- Automatic refresh token flow
- Retry failed request after token refresh
- Queue handling for multiple failed requests during refresh

UI / UX features

Depending on how far you implement the UI, this project can also cover:

- responsive layout
 - reusable table component
 - confirmation dialog / modal
 - empty state handling
 - success/error toast messages
 - loading spinner / skeleton
 - pagination controls
 - filter panel
-

2. Backend (.NET Core Web API) Technical Points Covered

ASP.NET Core Web API fundamentals

- Creating REST APIs in .NET Core / ASP.NET Core
- Controller-based API development
- API routing using attribute routing
- GET , POST , PUT , DELETE endpoints
- Model binding using [FromBody] , route params, query params
- Returning proper HTTP status codes

- API response standardization using common response wrapper

Clean project structure / layered architecture

The project covers a proper layered architecture such as:

- **Controllers**
- **Services**
- **Repositories**
- **DTOs**
- **Models / Entities**
- **Validators**
- **Middleware / Extensions / Configuration**
- **Stored Procedures / SQL scripts**

This is a very good interview/project architecture pattern.

CRUD API implementation

- Create Employee API
- Get all employees API
- Get employee by ID API
- Update employee API
- Delete employee API
- Search/filter/paging APIs
- master data APIs (e.g., departments) if added

DTO-based API design

- Separate request and response DTOs
- Avoid exposing DB models directly
- Use DTOs for:
 - create request
 - update request
 - login/register request
 - paged employee response
 - auth response
 - refresh token request
 - logout request

Service layer

- Business logic separated from controllers
- Validation of business rules
- Duplicate checks
- Error throwing / handling at service layer
- Auth logic separated into auth service

Repository pattern

- Repository interfaces and implementations
 - DB access abstracted away from controller/service
 - Reusable data access methods
 - Easier unit testing and maintainability
-

3. Database / SQL Server Technical Points Covered

SQL Server schema design

- Employees table
- Users table
- RefreshTokens table
- optionally Departments table
- primary keys and foreign keys
- audit columns if added (CreatedAt , UpdatedAt , etc.)

SQL concepts used

- IDENTITY
- primary key
- foreign key
- default values
- computed column (IsActive in refresh token table)
- nullable and non-nullable columns
- unique constraints if you add email/username uniqueness at DB level

Stored procedure based CRUD

If you follow the advanced version, this project covers:

- stored procedure for insert
- stored procedure for update
- stored procedure for delete
- stored procedure for paged employee listing
- optional stored procedure for search/sort/filter

Query design concepts

- filtering with optional parameters
- searching by name/email/department
- sorting dynamically
- pagination using:
 - OFFSET FETCH
 - OR ROW_NUMBER()
- count query for total records
- joins if department master is added

Real-world SQL practices

- checking duplicate email / duplicate username
 - token persistence in DB
 - token revocation handling
 - lookup/master table design
 - reusable stored procedures for enterprise style APIs
-

4. ADO.NET / Dapper / Data Access Technical Points Covered

Depending on the version you implemented, this project can cover **ADO.NET** or **Dapper**.

Core data access topics

- opening SQL connection using connection string
- using `SqlConnection`
- executing parameterized SQL
- calling stored procedures
- reading single row / multiple rows
- mapping query result to model/DTO
- insert/update/delete execution
- scalar result handling
- async DB access

If using Dapper

- `QueryAsync<T>()`
- `QueryFirstOrDefaultAsync<T>()`
- `ExecuteAsync()`
- passing parameters to SQL/stored procedures
- lightweight ORM usage

Secure DB coding practices

- parameterized queries
 - avoiding SQL injection
 - proper null handling
 - repository abstraction over DB access
-

5. Authentication & Authorization Technical Points Covered

This project actually becomes very strong here once Option 2 is completed.

Authentication

- User registration API

- User login API
- password hashing using **BCrypt**
- password verification
- JWT token generation
- JWT claims creation
- token expiry handling

JWT concepts covered

- access token generation
- issuer, audience, signing key
- claim types:
 - user id
 - username
 - role
- JWT validation middleware
- Bearer token authorization in API calls

Refresh token implementation

This is a strong real-world point.

Covered topics:

- refresh token generation
- secure random token creation
- storing refresh tokens in DB
- refresh token expiry
- refresh token validation
- refresh token rotation
- revoking old refresh token
- logout by revoking refresh token
- refresh token based silent login continuation

Authorization

- protecting employee APIs using `[Authorize]`
 - role-based authorization if you add role restrictions
 - frontend protected routes
 - redirect unauthorized users to login page
-

6. Validation Technical Points Covered

Backend validation

- `FluentValidation`

- create employee validation
- update employee validation
- login/register validation
- refresh token request validation
- logout request validation

Validation rules examples

- required fields
- max length
- email format
- salary > 0
- role required
- password minimum length
- custom business rule validation at service layer

Frontend validation

Depending on implementation:

- required fields in React forms
 - input-level validation message display
 - preventing invalid API calls from UI
-

7. API Security / Production-Oriented Technical Points Covered

Security practices

- password hashing using BCrypt
- JWT authentication
- refresh token storage in DB
- refresh token revocation on logout
- refresh token rotation
- not exposing password hash in responses
- protected API endpoints

Good API practices

- standardized API responses
- centralized exception handling middleware
- proper HTTP status codes:
 - 200
 - 201
 - 400
 - 401
 - 404

- 409
- 500
- DTO usage to avoid overexposing internal models

Production-style auth flow

- short-lived access token
 - long-lived refresh token
 - auto refresh on 401
 - logout cleanup
 - DB-backed token lifecycle
-

8. Error Handling / Middleware Technical Points Covered

Backend

- custom exception middleware
- global error handling
- returning consistent error response format
- handling validation exceptions
- handling unauthorized access exceptions
- handling not found / invalid operation cases

Frontend

- API error message display
 - handling unauthorized response
 - redirect on token expiry when refresh fails
 - catching and displaying CRUD operation errors
-

9. API Design / Enterprise Patterns Covered

API design patterns

- RESTful route naming
- separation of auth and employee endpoints
- pagination response structure
- common response wrapper
- query-based filtering model
- request/response DTO contract design

Enterprise coding style patterns

- layered architecture
- repository pattern

- service layer pattern
 - dependency injection
 - configuration binding from `appsettings.json`
 - extension methods for service registration
 - clean folder structure
-

10. Configuration / App Setup Technical Points Covered

Backend configuration

- `appsettings.json`
- connection string configuration
- JWT settings configuration
- options pattern / strongly typed settings class
- DI registrations in startup/program/service extension

Frontend configuration

- Axios base API URL
 - environment-based API URL if added
 - token storage utilities
 - reusable constants for routes/endpoints
-

11. Advanced Employee Module Features Covered (if you continue full Option 2)

If you continue the next parts of the advanced version, the project can also cover:

Employee listing enhancements

- server-side pagination
- server-side searching
- server-side sorting
- multi-filter employee search
- department-based filtering
- reusable paged response DTO

Master-detail design

- Department master table
- employee linked to department
- dropdown binding from API

Export features

- Excel export
- filtered export
- maybe PDF export later if you want

Audit / admin features (possible extension)

- created by / updated by
 - created date / modified date
 - soft delete vs hard delete discussion
 - activity logs
-

12. Dev / Tooling / Working Style Points Covered

Development tooling

- VS Code for full-stack development
- npm scripts for React
- .NET CLI commands
- Swagger for API testing
- SQL Server for DB
- Git-friendly project structure

API testing

- Swagger / OpenAPI usage
 - Postman style request testing
 - auth token testing
 - refresh token flow testing
-

13. Strong Interview Topics This Project Gives You

If someone asks “What concepts have you used in this project?”, you can talk about all of these:

React side

- hooks (`useState` , `useEffect`)
- component reusability
- form handling
- Axios interceptors
- protected routes
- token-based authentication in frontend
- pagination/search/sorting UI

.NET side

- layered architecture

- repository pattern
- service layer
- DTO mapping
- FluentValidation
- JWT auth
- refresh token implementation
- exception middleware
- dependency injection
- configuration using options pattern

SQL side

- CRUD queries / stored procedures
 - joins, filters, paging queries
 - refresh token persistence
 - schema design and constraints
-

14. Resume-Friendly Technical Stack Summary for This Project

If you want to describe this project technically, it covers something like this:

Frontend

- React
- Vite
- React Router
- Axios
- JavaScript / JSX
- localStorage based auth persistence
- interceptor-based token refresh flow

Backend

- ASP.NET Core Web API
- C#
- JWT Authentication
- Refresh Token Authentication
- FluentValidation
- Repository Pattern
- Service Layer Architecture
- Dependency Injection
- Exception Middleware

Database

- SQL Server

- Dapper / ADO.NET
 - Stored Procedures
 - Pagination / Search / Filter queries
 - relational schema design
-

15. If you want the “big picture” answer in one line

This project covers:

Full-stack CRUD development using React + ASP.NET Core Web API + SQL Server, including layered backend architecture, JWT + refresh token authentication, validation, repository/service patterns, protected routes, API integration with Axios interceptors, and production-style employee management features like paging, filtering, sorting, and secure auth flow.

16. My suggestion: keep this as 4 categories in your notes

If you're using this project for interview prep or resume, organize the covered points like this:

A. Frontend

- React CRUD
- Routing
- Protected routes
- Axios integration
- token storage
- interceptor refresh flow

B. Backend

- REST API
- layered architecture
- service/repository pattern
- DTOs
- validation
- middleware

C. Authentication

- BCrypt password hashing
- JWT
- refresh tokens
- token rotation
- logout revoke flow

D. Database

- SQL Server tables
- stored procedures / queries
- pagination / filtering / sorting
- refresh token persistence